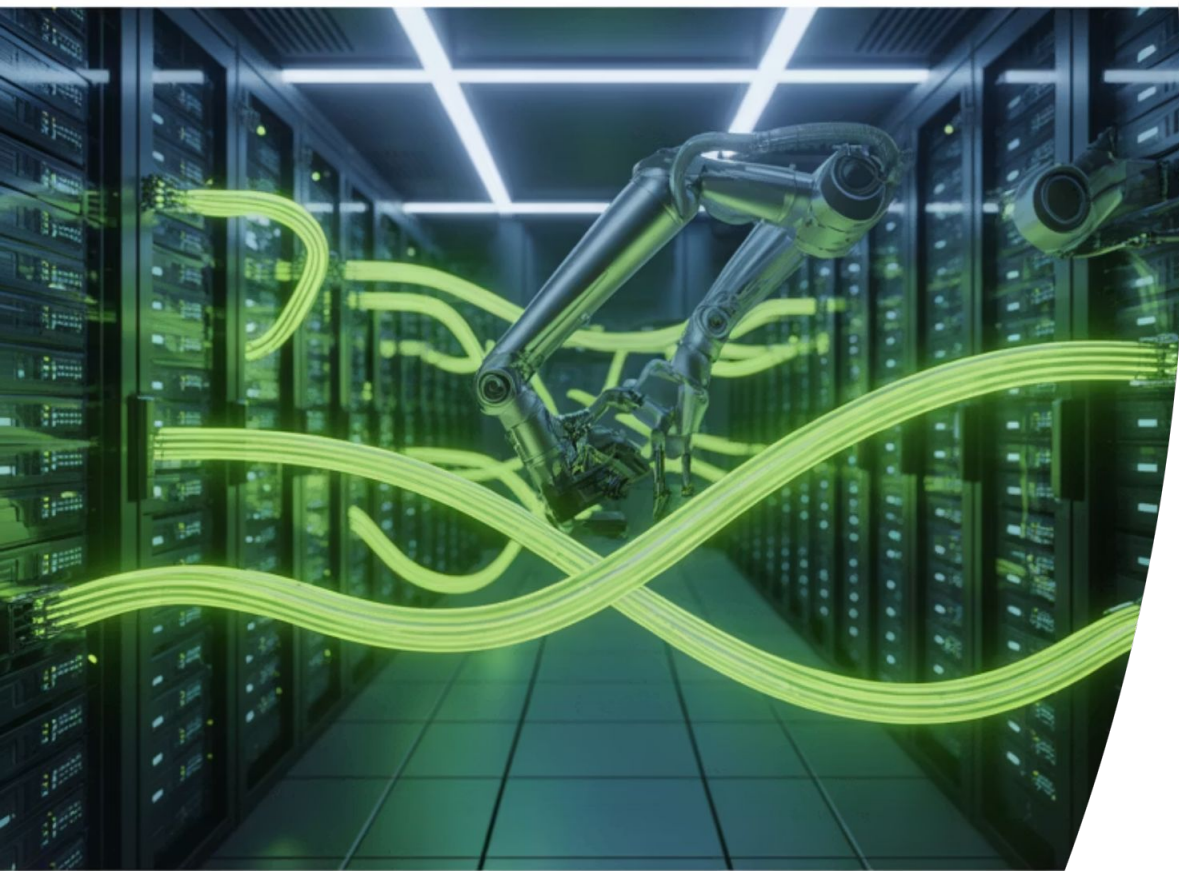




Les Playbooks Ansible et le langage YAML

Formation pratique complète pour maîtriser l'automatisation
avec Ansible

Pourquoi Ansible et YAML ?

Ansible est devenu l'outil de référence pour l'automatisation des infrastructures IT modernes. Sa force réside dans sa simplicité : pas d'agent à installer, une syntaxe lisible basée sur YAML, et une philosophie déclarative qui permet de décrire l'état souhaité plutôt que les étapes pour y parvenir.

Le langage YAML (Yet Another Markup Language) est au cœur d'Ansible. Conçu pour être facilement lisible par les humains, il permet de structurer des configurations complexes de manière claire et maintenable. Contrairement aux scripts shell traditionnels, un playbook Ansible décrit ce que vous voulez obtenir, pas comment l'obtenir.

Cette formation vous guidera à travers les fondamentaux du YAML, la structure des playbooks, les modules essentiels, et vous mènera jusqu'à la création de playbooks complets prêts pour la production. Vous découvrirez également le concept crucial d'idempotence, garantissant que vos automatisations sont sûres et prévisibles.

Ce que vous allez apprendre

Fondamentaux YAML

Maîtriser la syntaxe YAML, comprendre les structures de données (listes, dictionnaires), et éviter les pièges courants liés à l'indentation et aux types de données.

Structure des Playbooks

Comprendre l'anatomie complète d'un playbook Ansible, depuis les en-têtes de déclaration jusqu'à l'organisation des tâches et la gestion des privilèges.

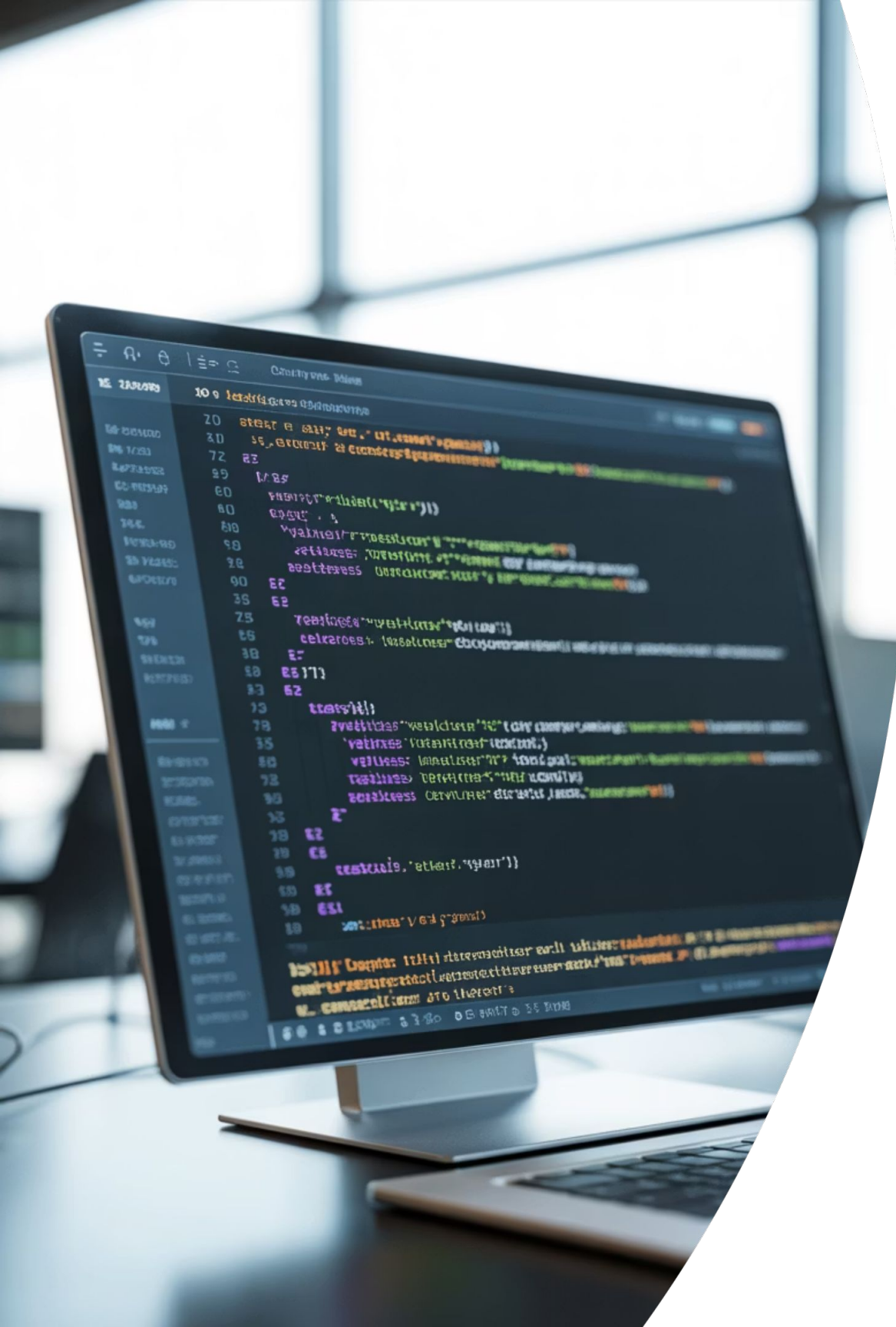
Modules Essentiels

Découvrir et pratiquer les modules Ansible les plus utilisés pour la gestion de paquets, services, fichiers et configurations système.

Variables & Facts

Apprendre à utiliser les variables pour rendre vos playbooks flexibles et réutilisables, et exploiter les facts pour adapter vos configurations.

À la fin de cette formation, vous serez capable de créer des playbooks Ansible professionnels, maintenables et idempotents, prêts à automatiser vos infrastructures en production.

A computer monitor is shown in the foreground, displaying a code editor with YAML code. The code is color-coded and includes comments. The monitor is on a desk, and a laptop is partially visible in the foreground. The background is a bright, out-of-focus window.

CHAPITRE 1

Le langage YAML

Fondamentaux et règles de syntaxe

YAML : Yet Another Markup Language

Qu'est-ce que YAML ?

YAML est un format de sérialisation de données conçu pour être facilement lisible par les humains. Contrairement au JSON ou au XML, YAML privilégie la clarté et la simplicité. Il est largement utilisé dans les fichiers de configuration, particulièrement dans l'écosystème DevOps. Le format YAML repose sur trois principes fondamentaux :

- L'indentation définit la structure (pas d'accolades ou de crochets encombrants)
- Les données sont organisées en paires clé-valeur
- La syntaxe est minimale et intuitive

Pourquoi YAML pour Ansible ?

Ansible a choisi YAML pour plusieurs raisons stratégiques qui facilitent l'adoption et la maintenance des playbooks :

Lisibilité : Un playbook se lit comme une checklist, compréhensible même par des non-développeurs

Maintenance : La structure claire facilite les modifications et le versioning

Collaboration : Les équipes peuvent facilement réviser et comprendre le code des autres

Moins d'erreurs : La syntaxe simple réduit les bugs de syntaxe

Règle n° 1 : L'indentation est CRUCIALE

Code YAML

```
# Correct - indentation de 2 espaces
serveur:
  nom: web-01
  ip: 192.168.1.10
  services:
    - nginx
    - php-fpm

# Incorrect - indentation incohérente
serveur:
  nom: web-01
  ip: 192.168.1.10  # 3 espaces
  services:
    - nginx # 1 espace
```

Explication

L'indentation en YAML n'est pas qu'une question de style : elle définit la hiérarchie et la structure des données. Contrairement à Python où l'indentation est syntaxique mais flexible, YAML exige une cohérence absolue.

Règles d'or de l'indentation :

Utilisez **toujours 2 espaces** par niveau d'indentation (standard Ansible)

- Ne mélangez JAMAIS espaces et tabulations
- Configurez votre éditeur pour convertir les tabs en espaces
- Chaque niveau enfant doit être indenté de manière cohérente



Astuce Pro : Dans VS Code, installez l'extension "YAML" qui valide automatiquement votre indentation et signale les erreurs en temps réel.

Les structures de données : Scalaires

Code YAML

```
# Chaînes de caractères
nom: "Serveur Production"
description: Serveur web principal
commentaire: 'Utilise des guillemets simples'

# Nombres
port: 8080
timeout: 30
ratio: 0.85

# Booléens (plusieurs syntaxes)
actif: true
debug: false
maintenance: yes
backup: no

# Null
ancien_ip: null
ancien_ip: ~
```

Explication

Les scalaires sont les types de données de base en YAML. Ils représentent une valeur unique (par opposition aux listes ou dictionnaires).

Chaînes de caractères :

- Guillemets optionnels pour les chaînes simples
- Guillemets doubles pour échapper des caractères spéciaux
- Guillemets simples pour du texte brut (pas d'échappement)

Nombres : Entiers et décimaux reconnus automatiquement, pas de guillemets nécessaires.

Booléens : YAML accepte plusieurs syntaxes : true/false, yes/no, on/off. Privilégiez true/false pour la clarté.

Valeurs nulles : Représentées par null ou le tilde ~. Utilisé pour des valeurs non définies ou optionnelles.

Les structures de données : Listes

Code YAML

```
# Syntaxe avec tirets (recommandée)
paquets:
  - nginx
  - php-fpm
  - mysql-server
  - redis

# Syntaxe inline (compacte)
ports: [80, 443, 8080]

# Liste d'objets complexes
utilisateurs:
  - nom: alice
    role: admin
    email: alice@example.com
  - nom: bob
    role: user
    email: bob@example.com

# Liste imbriquée
infrastructure:
  - serveurs:
      - web-01
      - web-02
    type: production
```

Explication

Les listes (ou séquences) en YAML permettent de regrouper plusieurs valeurs du même type. Elles sont essentielles dans Ansible pour définir des tâches, des hôtes, ou des packages à installer.

Syntaxe avec tirets :

- Chaque élément commence par un tiret suivi d'un espace
- Tous les éléments doivent avoir la même indentation
- Privilégiée pour la lisibilité dans les playbooks

Syntaxe inline : Utile pour des listes courtes et simples, utilise des crochets comme en JSON.

Listes d'objets : Chaque élément de liste peut être un dictionnaire complet, permettant de structurer des données complexes comme des configurations d'utilisateurs ou de serveurs.



Important : Le tiret doit être suivi d'un espace. "- nginx" est correct, "-nginx" générera une erreur.

Les structures de données : Dictionnaires

Code YAML

```
# Dictionnaire simple
serveur:
  hostname: web-prod-01
  ip: 192.168.1.100
  datacenter: paris

# Dictionnaire imbriqué
application:
  nom: MonApp
  version: 2.1.0
  base_donnees:
    host: db.example.com
    port: 5432
    nom: production_db
  cache:
    type: redis
    host: cache.example.com

# Dictionnaire inline (rare)
user: {name: admin, uid: 1000}

# Combinaison liste + dictionnaire
serveurs:
  - hostname: web-01
    ip: 192.168.1.10
  - hostname: web-02
    ip: 192.168.1.11
```

Explication

Les dictionnaires (ou mappings) sont des collections de paires clé-valeur. Ils constituent la structure la plus courante dans les playbooks Ansible, utilisés pour définir les paramètres des tâches, les variables, et les configurations.

Structure de base :

- Format "clé: valeur" avec un deux-points suivi d'un espace
- Les clés enfants sont indentées sous leur parent
- Les clés doivent être uniques dans un même dictionnaire

Imbrication : Les dictionnaires peuvent contenir d'autres dictionnaires à plusieurs niveaux, permettant de modéliser des configurations complexes comme des paramètres d'application avec sous-sections.

Combinaison avec listes : La vraie puissance vient de la combinaison listes + dictionnaires, permettant de définir des collections d'objets configurés (serveurs, utilisateurs, services, etc.).

Les pièges courants du

VAMI

Piège n° 1 : Les deux-points dans les chaînes

#

Bonnes pratiques YAML pour Ansible

Checklist de qualité

- Toujours utiliser 2 espaces d'indentation
- Commencer chaque fichier par trois tirets (---)
- Ajouter des commentaires pour les sections complexes
- Grouper logiquement les éléments liés
- Utiliser des noms de clés descriptifs et cohérents
- Privilégier la syntaxe multi-lignes pour la lisibilité
- Valider avec yamllint avant de commiter

Exemple de fichier bien structuré

```
---
# Configuration du serveur web production
# Auteur: Équipe DevOps
# Dernière mise à jour: 2024

# Définition des hôtes cibles
hosts: webservers
become: yes

# Variables globales
vars:
  nginx_version: "1.20"
  document_root: /var/www/html

# Tâches d'installation et configuration
tasks:
  - name: Installation de Nginx
    apt:
      name: nginx
      state: present

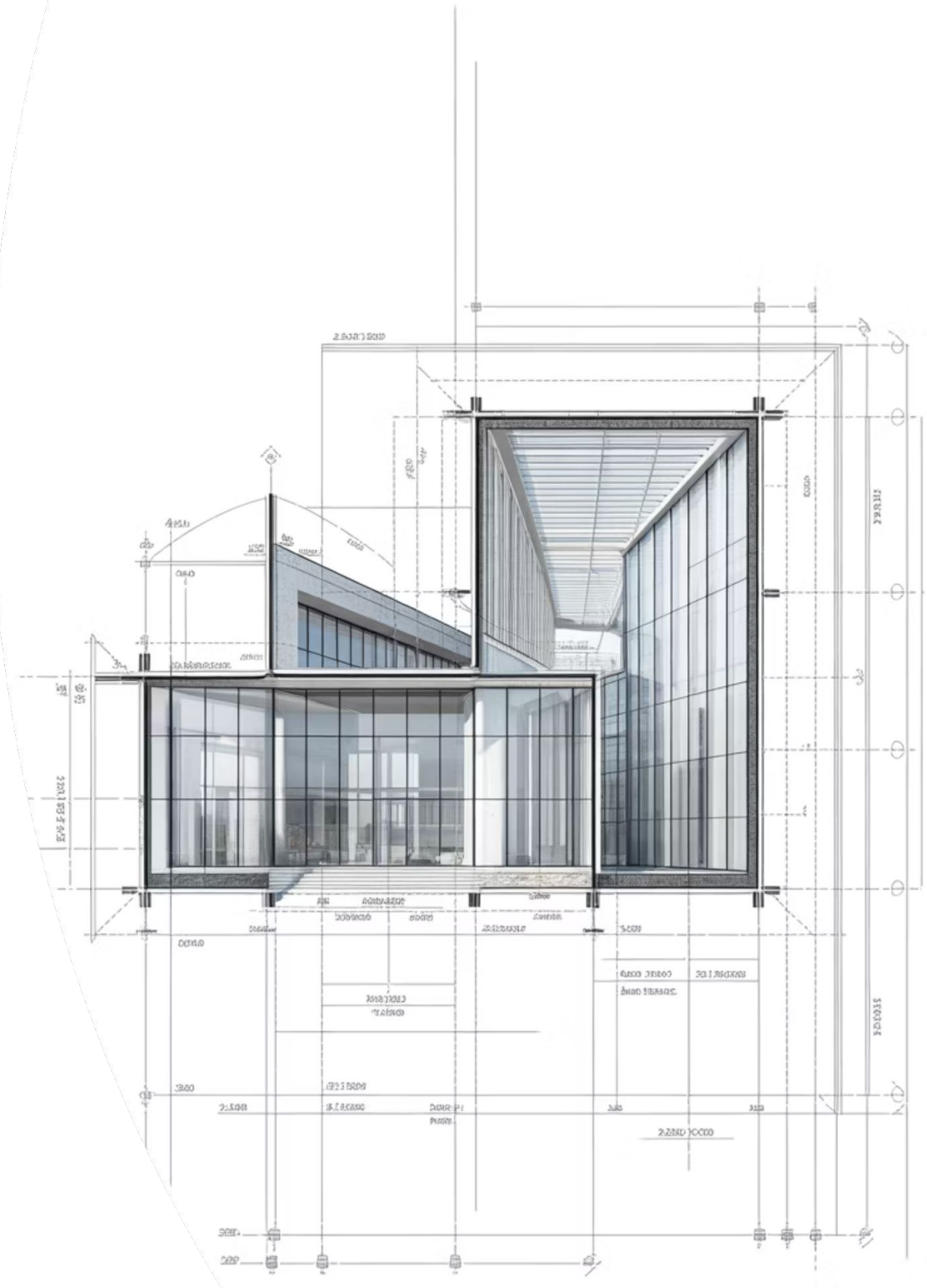
  - name: Configuration du site
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
```

Un fichier YAML bien organisé se lit comme un document : titres clairs, sections logiques, et commentaires pertinents.

CHAPITRE 2

Structure d'un Playbook Ansible

Anatomie complète d'un playbook professionnel



Anatomie d'un Playbook Ansible

Structure complète

```
---
# En-tête du playbook
- name: Configuration serveur web
  hosts: webserver
  become: yes
  gather_facts: yes

# Variables du playbook
vars:
  http_port: 80
  max_clients: 200

# Tâches à exécuter
tasks:
  - name: Installer Nginx
    apt:
      name: nginx
      state: present

  - name: Démarrer le service
    service:
      name: nginx
      state: started
      enabled: yes
```

Explication

Un playbook Ansible est un fichier YAML qui décrit une série d'actions à exécuter sur des hôtes distants. Il se compose de plusieurs sections essentielles, chacune ayant un rôle spécifique.

Les trois tirets (---) : Marqueur standard YAML indiquant le début d'un document. Bien que techniquement optionnel, c'est une bonne pratique de toujours l'inclure.

Le play : Niveau racine du playbook (liste commençant par un tiret). Un playbook peut contenir plusieurs plays pour différents groupes d'hôtes.

L'en-tête : Définit les paramètres globaux du play : nom descriptif, hôtes cibles, privilèges, collecte de faits.

Les variables : Section optionnelle définissant des valeurs réutilisables dans tout le playbook.

Les tâches : Cœur du playbook, liste ordonnée des actions à exécuter séquentiellement.

L'attribut "name" : Documentation

vivante

Code

```
---
```

```
#
```

L'attribut "hosts" : Cibler les bonnes machines

Code

```
---
# Inventaire (inventory.ini)
[webservers]
web-01.example.com
web-02.example.com

[databases]
db-01.example.com
db-02.example.com

[production:children]
webservers
databases

# Playbooks avec différentes cibles
- name: Config serveurs web
  hosts: webservers

- name: Config toutes les machines
  hosts: all

- name: Config une seule machine
  hosts: web-01.example.com

- name: Config production
  hosts: production

- name: Pattern avancé
  hosts: webservers:!web-02.example.com
```

Explication

L'attribut `hosts` définit sur quelles machines le playbook sera exécuté. Il fait référence à votre fichier d'inventaire Ansible qui liste toutes vos machines.

Types de cibles :

Groupe : `hosts: webservers` exécute sur toutes les machines du groupe `[webservers]`

all : `hosts: all` cible toutes les machines de l'inventaire

Machine unique : `hosts: web-01.example.com` cible un seul hôte

Groupe de groupes : `hosts: production` si vous avez défini `[production:children]`

Patterns avancés :

`webservers:databases` : Union (webservers OU databases)

`webservers:&production` : Intersection (webservers ET production)

`webservers:!web-02` : Exclusion (tous les webservers SAUF web-02)



Sécurité : Testez toujours avec `--check` et `--limit` avant d'exécuter sur la production !

L'attribut "become" : Élévation de privilèges

Code

```
---
# Niveau play - tous les tasks en sudo
- name: Installation packages système
  hosts: webserver
  become: yes
  tasks:
    - name: Installer Nginx
      apt:
        name: nginx
        state: present

# Niveau task - élévation sélective
- name: Configuration mixte
  hosts: webserver
  tasks:
    - name: Copier fichier utilisateur
      copy:
        src: config.txt
        dest: /home/user/config.txt

    - name: Installer package système
      apt:
        name: htop
        state: present
        become: yes

# Options avancées
- name: Config avec utilisateur spécifique
  hosts: database
  become: yes
  become_user: postgres
  become_method: sudo
```

Explication

L'attribut `become` permet d'élever les privilèges lors de l'exécution des tâches, équivalent à utiliser `sudo` en ligne de commande. C'est essentiel pour les opérations système comme installer des packages, modifier des fichiers de configuration système, ou gérer des services.

Niveaux d'application :

Niveau play : `become: yes` sous le nom du play applique l'élévation à toutes les tâches

Niveau task : `become: yes` dans une tâche spécifique pour un contrôle granulaire

Options complémentaires :

`become_user` : Spécifie l'utilisateur cible (par défaut `root`). Utile pour `postgres`, `mysql`, etc.

`become_method` : Méthode d'élévation (`sudo`, `su`, `pbrun`, etc.). Par défaut : `sudo`

`become_flags` : Flags additionnels passés à la méthode d'élévation

Bonnes pratiques : Utilisez l'élévation uniquement quand nécessaire. Pour des opérations utilisateur standard, ne pas utiliser `become` améliore la sécurité.

L'attribut "gather_facts" : Collecter les informations système

Code

```
---
# Activation de la collecte (défaut)
- name: Config avec facts
  hosts: webservers
  gather_facts: yes
  tasks:
    - name: Afficher l'OS détecté
      debug:
        msg: "OS: {{ ansible_distribution }} {{ ansible_distribution_version }}"

    - name: Installer selon l'OS
      apt:
        name: nginx
        state: present
        when: ansible_os_family == "Debian"

# Désactivation pour performance
- name: Tâche simple sans facts
  hosts: webservers
  gather_facts: no
  tasks:
    - name: Ping simple
      ping:

# Collecte manuelle sélective
- name: Collecte optimisée
  hosts: webservers
  gather_facts: no
  tasks:
    - name: Collecter seulement réseau
      setup:
        gather_subset:
          - network
```

Explication

L'attribut `gather_facts` contrôle si Ansible doit collecter automatiquement des informations sur les hôtes cibles avant d'exécuter les tâches. Par défaut, il est à `yes`.

Qu'est-ce qu'un fact ? Un fact est une information système collectée automatiquement : OS, version, architecture, adresses IP, disques, mémoire, etc. Ces données sont accessibles via des variables comme `ansible_distribution` ou `ansible_hostname`.

Utilité des facts :

Adaptabilité : Écrire des playbooks qui s'adaptent à différents OS ou configurations

Conditionnalité : Exécuter des tâches selon l'environnement détecté

Templating : Injecter des informations système dans les fichiers de configuration

Quand désactiver :

- Playbooks très simples qui n'utilisent pas de facts
- Optimisation de performance sur de nombreux hôtes
- La collecte ajoute 2-5 secondes par hôte

Collecte sélective : Utilisez le module `setup` avec `gather_subset` pour ne collecter que ce dont vous avez besoin.

La section "tasks" : Le cœur du playbook

Code

```
---
- name: Configuration serveur web
  hosts: webserver
  become: yes

  tasks:
    - name: Mettre à jour le cache APT
      apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Installer les packages requis
      apt:
        name:
          - nginx
          - python3-pip
          - git
        state: present

    - name: Copier le fichier de configuration
      copy:
        src: files/nginx.conf
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
      notify: Redémarrer Nginx

    - name: Démarrer et activer Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

Explication

La section `tasks` contient la liste ordonnée des actions à exécuter. C'est le cœur opérationnel du playbook. Les tâches sont exécutées séquentiellement, dans l'ordre de déclaration.

Structure d'une tâche :

name : Description lisible de l'action (fortement recommandé)

module : Le module Ansible à utiliser (apt, copy, service, etc.)

paramètres : Arguments spécifiques au module sous forme de dictionnaire

directives : Options comme notify, when, register, etc.

Ordre d'exécution : Les tâches s'exécutent de haut en bas. Si une tâche échoue, Ansible arrête l'exécution pour cet hôte (sauf si configuré autrement).

Notification de handlers : La directive `notify` déclenche un handler (action conditionnelle) seulement si la tâche modifie quelque chose. Dans l'exemple, Nginx ne redémarre que si le fichier de config change.

Idempotence : Chaque tâche doit être idempotente : exécutée plusieurs fois, elle produit le même résultat sans effets secondaires.

CHAPITRE 3

Les Modules Essentiels

Maîtriser les modules les plus utilisés dans la production



Module APT : Gestion de packages Debian/Ubuntu

Code

```
---
# Installation simple
- name: Installer Nginx
  apt:
    name: nginx
    state: present

# Installation multiple
- name: Installer plusieurs packages
  apt:
    name:
      - nginx
      - postgresql
      - redis-server
    state: present
    update_cache: yes

# Version spécifique
- name: Installer version précise
  apt:
    name: nginx=1.18.0-0ubuntu1
    state: present

# Désinstallation
- name: Supprimer Apache
  apt:
    name: apache2
    state: absent
    purge: yes

# Mise à jour complète
- name: Upgrade tous les packages
  apt:
    upgrade: dist
    update_cache: yes
```

Explication

Le module `apt` gère les packages sur les systèmes Debian et Ubuntu. Il utilise le gestionnaire de packages APT sous le capot, mais de manière idempotente et déclarative.

Paramètres essentiels :

name : Nom du package ou liste de packages à gérer

state : État souhaité (present, absent, latest)

update_cache : Rafraîchir la liste des packages (équivalent à `apt update`)

cache_valid_time : Durée en secondes de validité du cache (évite les updates répétés)

États disponibles :

present : Installer si absent (version disponible)

absent : Désinstaller si présent

latest : Installer la dernière version disponible

Options de désinstallation :

purge: yes : Supprime aussi les fichiers de configuration (comme `apt purge`)

autoremove: yes : Supprime les dépendances devenues inutiles



Astuce : Utilisez `cache_valid_time: 3600` pour éviter de rafraîchir le cache à chaque exécution.

Module YUM : Gestion de packages RedHat/CentOS

Code

```
---
# Installation simple
- name: Installer Nginx
  yum:
    name: nginx
    state: present

# Installation multiple
- name: Installer stack LAMP
  yum:
    name:
      - httpd
      - mariadb-server
      - php
      - php-mysql
    state: present

# Depuis un repo spécifique
- name: Installer depuis EPEL
  yum:
    name: httpd
    state: present
    enablerepo: epel

# Désinstaller
- name: Supprimer Apache
  yum:
    name: httpd
    state: absent

# Mettre à jour tous les packages
- name: Full system update
  yum:
    name: '*'
    state: latest
    update_cache: yes
```

Explication

Le module yum est l'équivalent de apt pour les distributions RedHat, CentOS, Fedora et leurs dérivés. Il offre des fonctionnalités similaires adaptées au gestionnaire de packages YUM/DNF.

Paramètres principaux :

name : Package(s) à gérer. Utilisez "*" pour tous les packages

state : present, absent, latest, installed, removed

update_cache : Rafraîchir les métadonnées des repos

enablerepo / disablerepo : Activer/désactiver des repos temporairement

Gestion des repositories :

enablerepo: epel : Active temporairement le repo EPEL pour cette installation

disablerepo: testing : Désactive temporairement un repo

- Utile pour installer depuis des repos non-standard sans modifier la config système

Note DNF : Sur les systèmes récents (Fedora 22+, RHEL 8+), YUM est remplacé par DNF. Ansible gère cette transition automatiquement, le module yum fonctionne avec les deux.

Alternative : Utilisez le module package pour un code agnostique de la distribution.

Module SERVICE : Gérer les services système

Code

```
---
# Démarrer un service
- name: Démarrer Nginx
  service:
    name: nginx
    state: started

# Démarrer et activer au boot
- name: Activer Nginx au démarrage
  service:
    name: nginx
    state: started
    enabled: yes

# Redémarrer un service
- name: Redémarrer après config
  service:
    name: nginx
    state: restarted

# Recharger la config (sans coupure)
- name: Recharger Nginx
  service:
    name: nginx
    state: reloaded

# Arrêter et désactiver
- name: Désactiver Apache
  service:
    name: apache2
    state: stopped
    enabled: no

# Vérifier le statut
- name: Vérifier état de PostgreSQL
  service:
    name: postgresql
    state: started
    check_mode: yes
```

Explication

Le module `service` gère les services système via systemd, SysV init, upstart ou autres gestionnaires de services. Il abstrait les différences entre distributions pour un code portable.

États disponibles :

started : Démarre le service s'il est arrêté (idempotent)

stopped : Arrête le service s'il tourne

restarted : Force un redémarrage complet du service

reloaded : Recharge la configuration sans arrêter le service (quand supporté)

Activation au boot :

enabled: yes : Le service démarre automatiquement au boot système

enabled: no : Désactive le démarrage automatique

- Indépendant de l'état actuel (started/stopped)

Différence restart vs reload :

restart : Arrêt complet puis redémarrage. Cause une interruption de service.

reload : Relecture de la config à chaud, sans coupure. Préférez-le quand disponible (nginx, apache supportent reload).



Bonne pratique : Combinez toujours `state: started` avec `enabled: yes` pour garantir que le service tourne ET redémarre au boot.

Module COPY : Transférer des fichiers

Code

```
---
# Copie simple
- name: Copier fichier de config
  copy:
    src: files/nginx.conf
    dest: /etc/nginx/nginx.conf

# Avec permissions et propriétaire
- name: Copier avec permissions
  copy:
    src: files/app.conf
    dest: /etc/app/app.conf
    owner: www-data
    group: www-data
    mode: '0644'

# Backup avant écrasement
- name: Copier avec backup
  copy:
    src: files/important.conf
    dest: /etc/important.conf
    backup: yes

# Contenu inline (petit fichier)
- name: Créer fichier avec contenu
  copy:
    content: |
      server {
        listen 80;
        server_name example.com;
      }
    dest: /etc/nginx/sites-available/example
    mode: '0644'

# Validation avant copie
- name: Copier et valider
  copy:
    src: files/nginx.conf
    dest: /etc/nginx/nginx.conf
    validate: 'nginx -t -c %s'
```

Explication

Le module `copy` transfère des fichiers depuis la machine de contrôle (celle qui exécute Ansible) vers les hôtes distants. C'est le module de base pour déployer des fichiers de configuration statiques.

Paramètres essentiels :

src : Chemin source sur la machine de contrôle (relatif au playbook ou absolu)

dest : Chemin destination sur l'hôte distant (toujours absolu)

owner / group : Propriétaire et groupe du fichier créé

mode : Permissions en notation octale (toujours entre guillemets : '0644')

Options de sécurité :

backup : `yes` : Crée une copie horodatée du fichier existant avant écrasement

validate : Commande de validation avant copie finale. Le `%s` est remplacé par le chemin du fichier temporaire. Si la validation échoue, le fichier n'est pas copié.

Contenu inline : Utilisez `content` au lieu de `src` pour créer des petits fichiers directement depuis le playbook. Utile pour des configs simples ou des fichiers générés.

Idempotence : Copy ne transfère que si le fichier a changé (checksum). Exécutions répétées = pas de transfert inutile.

Module FILE : Gérer fichiers et répertoires

Code

```
---
# Créer un répertoire
- name: Créer dossier application
  file:
    path: /var/www/myapp
    state: directory
    owner: www-data
    group: www-data
    mode: '0755'

# Créer répertoires parents
- name: Créer arborescence complète
  file:
    path: /opt/app/data/uploads
    state: directory
    recurse: yes
    mode: '0755'

# Créer un lien symbolique
- name: Lien vers config active
  file:
    src: /etc/nginx/sites-available/mysite
    dest: /etc/nginx/sites-enabled/mysite
    state: link

# Supprimer fichier ou dossier
- name: Supprimer fichier temporaire
  file:
    path: /tmp/oldfile.txt
    state: absent

# Modifier permissions récursivement
- name: Sécuriser répertoire
  file:
    path: /etc/myapp
    owner: root
    group: root
    mode: '0750'
    recurse: yes
```

Explication

Le module `file` gère les fichiers, répertoires et liens symboliques sur les hôtes distants. Contrairement à `copy`, il ne transfère pas de contenu mais gère les propriétés et l'existence des fichiers.

États disponibles :

directory : Crée un répertoire s'il n'existe pas

file : Vérifie qu'un fichier existe (sans le créer)

link : Crée un lien symbolique

hard : Crée un lien dur

absent : Supprime fichier ou répertoire

touch : Crée un fichier vide ou met à jour son timestamp

Gestion des répertoires :

recurse: yes : Applique owner/group/mode récursivement à tous les sous-éléments

Crée automatiquement les répertoires parents avec **state: directory**

Liens symboliques :

src : Cible du lien (chemin vers lequel le lien pointe)

dest : Emplacement du lien lui-même

- Utile pour activer des sites nginx/apache ou basculer entre versions

Cas d'usage courants : Créer des structures de répertoires, gérer les permissions, nettoyer des fichiers temporaires, activer/désactiver des configurations.

Comparaison des modules essentiels



APT / YUM

Usage : Gestion de packages système

Quand l'utiliser :

- Installer des logiciels (nginx, postgresql, etc.)
- Mettre à jour le système
- Désinstaller des packages

Particularité : APT pour Debian/Ubuntu, YUM pour RedHat/CentOS. Utilisez `package` pour du code universel.



SERVICE

Usage : Contrôle des services système

Quand l'utiliser :

- Démarrer/arrêter des services
- Activer au boot
- Redémarrer après modification de config

Particularité : Abstrait systemd/init/upstart. Préférez `reload` à `restart` quand possible.



COPY

Usage : Transfert de fichiers statiques

Quand l'utiliser :

- Déployer fichiers de configuration
- Transférer scripts ou binaires

Particularité : Pour fichiers statiques. Créer petits fichiers via content dynamiques (avec variables), utilisez plutôt `template`.



FILE

Usage : Gestion fichiers/dossiers/liens

Quand l'utiliser :

- Créer répertoires
- Modifier permissions/propriétaires

Particularité : Ne transfère pas de contenu, gère uniquement les métadonnées et l'existence.



CHAPITRE 4

Variables et Facts

Rendre vos playbooks flexibles et réutilisables

Introduction aux variables

Code

```
---
# Définir des variables
- name: Config avec variables
  hosts: webservers
  vars:
    http_port: 80
    max_clients: 200
    app_name: "MonApp"

tasks:
  - name: Afficher variables
    debug:
      msg: "{{ app_name }}" écoute sur port {{ http_port }}"

  - name: Utiliser dans module
    lineinfile:
      path: /etc/nginx/nginx.conf
      line: "worker_connections {{ max_clients }};"

# Variables dans fichier séparé
- name: Config avec vars externes
  hosts: webservers
  vars_files:
    - vars/common.yml
    - vars/production.yml

# Surcharge en ligne de commande
# ansible-playbook playbook.yml -e "http_port=8080"
```

Explication

Les variables permettent de rendre vos playbooks flexibles, réutilisables et maintenables. Au lieu de coder en dur des valeurs, vous les paramétrez pour faciliter les changements et l'adaptation à différents environnements.

Définition des variables :

Section vars : Dans le playbook, sous `vars :`

Fichiers externes : Via `vars_files` pointant vers des fichiers YAML

Inventaire : Variables par hôte ou par groupe dans l'inventaire

Ligne de commande : Avec `-e` ou `--extra-vars` (priorité maximale)

Utilisation des variables :

Syntaxe Jinja2 : `{{ nom_variable }}`

- Dans les valeurs de paramètres de modules
- Dans les templates de fichiers
- Dans les conditions et boucles

Bonnes pratiques :

- Noms descriptifs en `snake_case`
- Groupez les variables liées (`vars/database.yml`, `vars/web.yml`)
- Documentez les variables obligatoires

Les Facts : Informations système automatiques

Code

```
---
- name: Utiliser les facts
  hosts: all
  gather_facts: yes

tasks:
  # Afficher tous les facts disponibles
  - name: Voir tous les facts
    debug:
      var: ansible_facts

  # Utiliser des facts spécifiques
  - name: Afficher infos système
    debug:
      msg: |
        OS: {{ ansible_distribution }} {{ ansible_distribution_version }}
        Hostname: {{ ansible_hostname }}
        IP: {{ ansible_default_ipv4.address }}
        CPU: {{ ansible_processor_cores }} cores
        RAM: {{ ansible_memtotal_mb }} MB

  # Installation conditionnelle selon l'OS
  - name: Installer package selon OS
    package:
      name: "{{ 'httpd' if ansible_os_family == 'RedHat' else 'apache2' }}"
      state: present

  # Adapter config selon la mémoire
  - name: Config MySQL selon RAM
    lineinfile:
      path: /etc/mysql/my.cnf
      line: "innodb_buffer_pool_size = {{ (ansible_memtotal_mb * 0.7) | int }}M"
```

Explication

Les facts sont des variables spéciales automatiquement collectées par Ansible au début de l'exécution d'un playbook (sauf si `gather_facts: no`). Ils contiennent des informations détaillées sur chaque hôte distant.

Categories de facts :

Système d'exploitation : Distribution, version, famille OS

Matériel : CPU, mémoire, disques, architecture

Réseau : Interfaces, adresses IP, DNS

Environnement : Variables d'environnement, utilisateur, chemins

Facts les plus utilisés :

`ansible_distribution` : Ubuntu, CentOS, Debian...

`ansible_os_family` : Debian, RedHat, Windows...

`ansible_hostname` : Nom d'hôte de la machine

`ansible_default_ipv4.address` : Adresse IP principale

`ansible_memtotal_mb` : Mémoire RAM totale en MB

`ansible_processor_cores` : Nombre de cœurs CPU

Cas d'usage : Adapter automatiquement les configurations selon la mémoire disponible, choisir les bons packages selon l'OS, configurer des services selon l'architecture réseau.

Variables vs Facts : Quand utiliser quoi ?

Variables définies par l'utilisateur

Ce que c'est : Valeurs que VOUS définissez explicitement dans vos playbooks, fichiers vars, ou inventaire.

Quand utiliser :

- Paramètres spécifiques à votre application (ports, chemins, noms)
- Configuration qui varie entre environnements (dev, staging, prod)
- Valeurs que vous voulez pouvoir modifier facilement
- Secrets et credentials (via ansible-vault)

Exemples :

- `app_version: "2.1.0"`
- `database_host: "db.example.com"`
- `ssl_certificate_path: "/etc/ssl/certs/myapp.crt"`

Facts collectés automatiquement

Ce que c'est : Informations découvertes automatiquement par Ansible sur les hôtes distants.

Quand utiliser :

- Adapter automatiquement selon l'environnement système
- Écrire des playbooks multi-OS portables
- Optimiser automatiquement selon les ressources disponibles
- Générer des configurations dynamiques (templates)

Exemples :

`ansible_distribution` pour choisir le bon gestionnaire de packages

`ansible_memtotal_mb` pour dimensionner les buffers

`ansible_default_ipv4.address` pour configurer des bindings réseau

❏ **Bonne pratique :** Combinez les deux ! Utilisez des facts pour l'adaptation automatique et des variables pour les paramètres métier. Par exemple : `max_connections: "{{ (ansible_memtotal_mb / 10) | int if auto_tune else fixed_connections }}"`

CHAPITRE 5

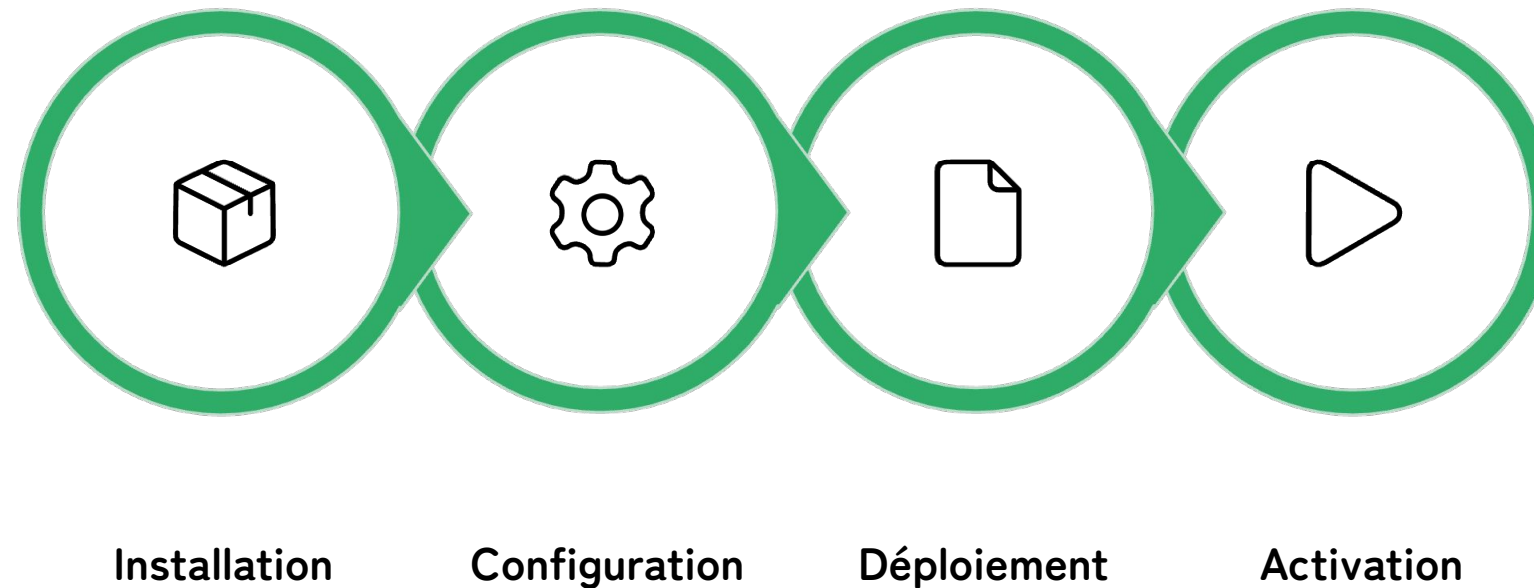
Démonstration Pratique

Playbook complet pour installer et configurer Nginx



nginx Web Server
Configuration
Deployment
Automation

Architecture de la démonstration



Notre playbook de démonstration illustre un workflow complet et réaliste : installation d'un package, configuration personnalisée, déploiement de fichiers, et activation du service. Chaque étape est idempotente et peut être exécutée plusieurs fois sans effets secondaires.

Cette approche en 4 phases est un pattern standard dans l'administration système automatisée. Elle garantit que chaque composant est en place avant de passer au suivant, tout en permettant de relancer le playbook à tout moment pour corriger ou mettre à jour la configuration.

Playbook complet : Vue d'ensemble

Code complet

```
---
- name: Installation et configuration Nginx
  hosts: webservers
  become: yes
  gather_facts: yes

  vars:
    nginx_port: 80
    server_name: example.com
    document_root: /var/www/html

  tasks:
    - name: Installer Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Créer le répertoire du site
      file:
        path: "{{ document_root }}"
        state: directory
        owner: www-data
        group: www-data
        mode: '0755'

    - name: Copier la configuration Nginx
      copy:
        content: |
          server {
            listen {{ nginx_port }};
            server_name {{ server_name }};
            root {{ document_root }};
            index index.html;
          }
        dest: /etc/nginx/sites-available/default
        backup: yes
        notify: Recharger Nginx

    - name: Copier la page d'accueil
      copy:
        dest: "{{ document_root }}/index.html"
        owner: www-data
        group: www-data
        mode: '0644'

    - name: Démarrer et activer Nginx
      service:
        name: nginx
        state: started
        enabled: yes
        handlers:
          - name: Recharger Nginx
            service:
              name: nginx
              state: reloaded
```

Bienvenue sur {{ server_name }}

Explication globale

Ce playbook représente un exemple complet et production-ready d'automatisation avec Ansible. Il démontre tous les concepts essentiels dans un cas d'usage réel.

Structure du playbook :

En-tête : Cible les webservers avec élévation de privilèges

Variables : Paramètres configurables (port, domaine, chemins)

Tasks : 5 tâches séquentielles couvrant l'installation complète

Handlers : Action conditionnelle (reload) déclenchée par notify

Points clés illustrés :

- Utilisation de plusieurs modules complémentaires
- Variables pour la flexibilité
- Gestion des permissions et propriétaires
- Backup automatique avant modification
- Handler pour recharger uniquement si nécessaire

Idempotence à chaque étape

Exécutez ce playbook plusieurs fois : la deuxième exécution ne changera rien (toutes les tâches retourneront "ok" au lieu de "changed"). C'est l'idempotence en action !

Étape 1 : Installation du package Nginx

Code

```
- name: Installer Nginx
  apt:
    name: nginx
    state: present
    update_cache: yes
```

Explication détaillée

La première tâche installe le serveur web Nginx en utilisant le gestionnaire de packages APT (pour Debian/Ubuntu).

Paramètres utilisés :

`name: nginx` : Spécifie le package à installer

`state: present` : Garantit que nginx est installé. Si déjà présent, ne fait rien (idempotence).

`update_cache: yes` : Rafraîchit la liste des packages avant installation (équivalent à `apt update`)

Comportement idempotent :

Première exécution : Installe nginx, retourne "changed"

Exécutions suivantes : Détecte que nginx est déjà installé, retourne "ok", ne fait rien

- Pas de réinstallation inutile, pas de modifications

Pourquoi `update_cache` ? Assure que vous installez la dernière version disponible. Sans cela, vous pourriez installer une version obsolète si le cache APT est ancien.



Alternative multi-OS : Utilisez le module `package` au lieu de `apt` pour un playbook fonctionnant sur Debian et RedHat.

Étape 2 : Création du répertoire du site

Code

```
vars:
  document_root: /var/www/html

tasks:
  - name: Créer le répertoire du site
    file:
      path: "{{ document_root }}"
      state: directory
      owner: www-data
      group: www-data
      mode: '0755'
```

Explication détaillée

Cette tâche garantit que le répertoire qui contiendra les fichiers du site web existe et a les bonnes permissions.

Utilisation de la variable :

`{{ document_root }}` : Référence la variable définie dans `vars`

- Avantage : Changez le chemin une seule fois dans `vars`, il se propage partout
- Facilite l'adaptation à différents environnements

Gestion des permissions :

`owner: www-data` : Propriétaire du répertoire (utilisateur sous lequel tourne nginx)

`group: www-data` : Groupe propriétaire

`mode: '0755'` : Permissions (rwxr-xr-x) permettant à nginx de lire et servir les fichiers

Idempotence :

- Si le répertoire existe déjà avec les bonnes permissions : "ok", rien ne change
- Si le répertoire existe mais avec de mauvaises permissions : "changed", permissions corrigées
- Si le répertoire n'existe pas : "changed", création avec les bonnes permissions

Étape 3 : Configuration de Nginx

Code

```
vars:
  nginx_port: 80
  server_name: example.com
  document_root: /var/www/html

tasks:
  - name: Copier la configuration Nginx
    copy:
      content: |
        server {
          listen {{ nginx_port }};
          server_name {{ server_name }};
          root {{ document_root }};
          index index.html;
        }
      dest: /etc/nginx/sites-available/default
      backup: yes
    notify: Recharger Nginx
```

Explication détaillée

Cette tâche crée le fichier de configuration Nginx avec du contenu dynamique généré à partir des variables.

Utilisation de content :

Au lieu de `src`, on utilise `content` pour générer le fichier inline

Le symbole `|` permet un contenu multi-lignes en YAML

Les variables sont interpolées via `{{ nom_variable }}`

Configuration Nginx générée :

`listen 80` : Nginx écoute sur le port spécifié dans `nginx_port`

`server_name` : Nom de domaine du site

`root` : Répertoire contenant les fichiers du site

`index` : Fichier à servir par défaut

Options de sécurité :

`backup: yes` : Avant d'écraser, crée une copie horodatée de l'ancien fichier de config

- Permet de rollback en cas de problème

Notification de handler :

`notify: Recharger Nginx` : Si la config change, déclenche le handler correspondant

- Le reload n'a lieu QUE si le fichier a réellement changé

Étape 4 : Déploiement de la page d'accueil

Code

```
vars:
  server_name: example.com
  document_root: /var/www/html

tasks:
  - name: Copier la page d'accueil
    copy:
      content: "
```

Bienvenue sur {{ server_name }}

```
" dest: "{{ document_root }}/index.html"  owner: www-data  group: www-data
mode: '0644'
```

Explication détaillée

Cette tâche crée une page HTML simple qui sera servie par Nginx. Dans un contexte réel, vous utiliseriez probablement `src` pour copier un vrai fichier, mais ici on illustre la génération dynamique.

Contenu dynamique :

Le HTML inclut la variable `{{ server_name }}`

- Si vous changez `server_name` dans `vars`, la page se met à jour automatiquement
- Démontre comment personnaliser du contenu selon l'environnement

Chemin de destination :

`dest: "{{ document_root }}/index.html"` : Combine la variable avec un sous-chemin

Résultat final : `/var/www/html/index.html`

Permissions adaptées : Les permissions sont nécessaires car la chaîne commence par une variable

`owner/group: www-data` : Nginx peut lire le fichier

`mode: '0644'` : Lecture pour tous, écriture pour le propriétaire (`rw-r--r--`)

- Plus restrictif que le répertoire (`0755`) car fichier, pas dossier

Idempotence : Même contenu généré à chaque exécution = checksum identique = pas de réécriture inutile.

Étape 5 : Démarrage et activation du service

Code

```
- name: Démarrer et activer Nginx
  service:
    name: nginx
    state: started
    enabled: yes

handlers:
  - name: Recharger Nginx
    service:
      name: nginx
      state: reloaded
```

Explication détaillée

Cette tâche finale garantit que Nginx tourne et démarrera automatiquement au boot du serveur. Le handler associé permet de recharger la configuration sans interruption de service.

Gestion du service :

`state: started` : Démarre nginx s'il est arrêté, ne fait rien s'il tourne déjà

`enabled: yes` : Configure nginx pour démarrer automatiquement au boot système

- Ces deux paramètres sont indépendants et complémentaires

Le pattern Handler :

- Les handlers sont des tâches spéciales qui ne s'exécutent que si notifiées

Déclenchés par `notify: Recharger Nginx` dans la tâche de configuration

- S'exécutent à la FIN du playbook, même si notifiés plusieurs fois (1 seule exécution)
- Parfait pour les redémarrages de services conditionnels

Reload vs Restart :

`reloaded` : Recharge la config à chaud sans couper les connexions existantes

- Nginx supporte le reload gracieux (contrairement à un restart brutal)
- Production-safe : pas de downtime lors des déploiements

Le concept d'Idempotence

Qu'est-ce que l'idempotence ?

L'idempotence est le principe fondamental d'Ansible : **exécuter un playbook plusieurs fois produit le même résultat que l'exécuter une seule fois**. Aucun effet secondaire, aucune modification inutile.

En pratique :

- Première exécution : Installation et configuration complètes (état "changed")
- Deuxième exécution : Ansible détecte que tout est déjà en place (état "ok")
- Aucune modification n'est effectuée lors de la deuxième exécution
- Le résultat final est identique

Pourquoi c'est crucial :

Sécurité : Relancer un playbook ne casse rien

Débogage : Vous pouvez réexécuter sans crainte lors du dépannage

Convergence : Le système converge vers l'état désiré, peu importe son état initial

Correction automatique : Si quelqu'un modifie manuellement une config, le playbook la rétablit

Exemple avec notre playbook Nginx

Première exécution :

```
TASK [Installer Nginx] ****
changed: [webserver-01]

TASK [Créer le répertoire] ****
changed: [webserver-01]

TASK [Copier la configuration] ****
changed: [webserver-01]

TASK [Copier la page] ****
changed: [webserver-01]

TASK [Démarrer Nginx] ****
```

Deuxième exécution immédiate :

```
TASK [Installer Nginx] ****
ok: [webserver-01]

TASK [Créer le répertoire] ****
ok: [webserver-01]

TASK [Copier la configuration] ****
ok: [webserver-01]

TASK [Copier la page] ****
ok: [webserver-01]

TASK [Démarrer Nginx] ****
ok: [webserver-01]
```

Toutes les tâches retournent "ok" : Ansible a détecté que l'état désiré est déjà atteint. Aucune action n'a été effectuée.

Idempotence : Comment ça marche techniquement ?

Modules APT/YUM

Vérification : Interroge le gestionnaire de packages pour savoir si le package est déjà installé et à la bonne version.

Action conditionnelle : N'installe que si absent ou si version incorrecte.

Résultat : "changed" uniquement lors d'une vraie installation/mise à jour.

Module COPY

Vérification : Calcule le checksum SHA256 du fichier source et du fichier destination (s'il existe).

Action conditionnelle : Ne copie que si les checksums diffèrent ou si le fichier n'existe pas.

Résultat : "changed" uniquement si le contenu a réellement changé.

Module SERVICE

Vérification : Interroge systemd/init pour connaître l'état actuel du service.

Action conditionnelle : Ne démarre que si arrêté, n'active que si désactivé au boot.

Résultat : "changed" uniquement lors d'un changement d'état réel.

Module FILE

Vérification : Compare les permissions, propriétaire, et groupe actuels avec ceux demandés.

Action conditionnelle : Ne modifie que si les attributs diffèrent, crée uniquement si absent.

Résultat : "changed" uniquement lors de création ou modification d'attributs.

Tous les modules Ansible officiels implémentent cette logique de vérification avant action. C'est ce qui rend Ansible déclaratif et safe. Vous décrivez l'état final désiré, Ansible se charge de déterminer quelles actions (si nécessaires) permettront de l'atteindre.

Récapitulatif

Ce que vous avez appris

01

Fondamentaux YAML

Syntaxe, structures de données (listes, dictionnaires), indentation, et pièges courants à éviter

03

Modules Essentiels

APT/YUM (packages), SERVICE (services), COPY (fichiers), FILE (répertoires et permissions)

05

Pratique Complète

Playbook de production pour Nginx : installation, configuration, déploiement, activation

02

Anatomie des

Playbooks (play, hosts, become), section tasks, organisation et bonnes pratiques de structure

04

Variables et Facts

Paramétrage flexible avec variables définies, collecte automatique d'informations système avec facts

06

Idempotence

Le principe clé : exécutions multiples = résultat identique, sécurité et fiabilité garanties